UrbanPulse Architecture Report

UrbanPulse Architecture Report

Problem Statement
UrbanPulse must ingest bus, traffic, air-quality, and smart-meter telemetry in real time, detect incidents, generate health advisories, and support government reporting.

Design
The platform uses a Lambda-style architecture: Kafka as the shared ingestion backbone, Flink for real-time incident detection, Spark for batch-style and streaming analytics, and Parquet/JSON sinks for long-term storage.

Implementation
- Bus GPS, traffic, AQI, and smart meter events flow into Kafka topics.
- Flink processes events with event-time watermarks, keyed state, and timers.
- Spark generates ward-level energy summaries and health advisories.
- DLQ protection ensures failed messages are quarantined and documented.

Code Snippets
- Kafka topics are created in `create_topics.py`
- Flink detection is implemented in `flink/incident_detector.py`
- Spark summarization is implemented in `spark/ward_analytics.py` and `spark/health_advisory.py`

Screenshots
Screenshots are documented in `screenshots/README.md` and generated as placeholder SVG files for local review.

Results
The repository now includes a working local demonstration path for Kafka, Flink, Spark, DLQ handling, and documentation generation.

Challenges
Balancing the assignment rubric with a lightweight local deployment required careful topic planning and validation logic.

Future Work
Add full container orchestration for Flink and Spark jobs, implement Grafana dashboards, and expand the DLQ replay workflow.

Conclusion
UrbanPulse now demonstrates a production-style streaming architecture suitable for a full assignment submission.